

AI-Based Delivery Optimizer

Adaptive Delivery Optimization Platform & Event-Driven Courier Routing and Disruption Management System

Professional End-to-End Project Plan

Senior Architecture & Engineering Blueprint

1. Executive Project Overview

What the Project Is

The AI-Based Delivery Optimizer is a discrete-event simulation system that models an urban last-mile delivery operation and demonstrates how four classical optimization algorithms interact to keep a courier fleet operating efficiently under continuous real-world disruption. The system simulates a full delivery shift — a road network, a courier fleet, an incoming order stream, and a configurable sequence of disruptions — and produces real-time visualizations of how each algorithm responds to each event, how solution quality evolves over the shift, and what the measurable trade-off between speed and optimality actually costs in kilometers and on-time delivery rate.

The project is built around one central engineering insight: in live delivery operations, a perfect solution computed slowly is worse than a good solution computed immediately. The system must manage that trade-off dynamically — using a fast greedy heuristic when disruptions require immediate response, and scheduling deeper dynamic programming re-optimization during quiet windows when computational budget is available. Every design decision, from the algorithm interface contract to the shadow route tracker, serves this comparison objective.

What the Project Is Not

It is not a machine learning project. There are no neural networks, no training pipelines, no probabilistic models. The "AI" in the title refers to intelligent algorithmic decision-making: the system reasons about trade-offs, selects algorithms dynamically, and adapts its behavior based on current operational context. This is classical AI in the original sense — search, optimization, and constraint satisfaction.

It is not a real-time distributed system. It is not a microservices architecture. It is not a cloud-scale platform. It is a simulation — a controlled, reproducible, single-threaded model of a delivery operation that runs fast-forward and can be paused, stepped, and replayed. Every millisecond in the simulation represents a unit of simulated time, not wall-clock time.

It is not a CRUD application. The backend's primary responsibility is running an optimization engine, not managing database records. There is no user authentication, no multi-tenancy, no persistent user data.

Problem Statement

Static route planning in last-mile delivery fails in production because the plan computed at shift start diverges from reality within minutes. New orders arrive that were not in the original plan. Roads close. Vehicles break down. Traffic builds on key arteries. Every one of these disruptions invalidates some portion of the existing plan and requires a new decision. A human dispatcher cannot re-optimize six courier routes simultaneously across 50 active orders in under a minute. An algorithm can, but choosing the wrong algorithm produces either unacceptably slow responses or unacceptably poor route quality. The system this project builds makes that trade-off explicit, measurable, and controllable.

Main Objectives

The primary objective is to demonstrate the trade-off between solution quality and computational cost across four algorithm types under realistic disruption conditions. This means not just implementing the algorithms, but measuring them, comparing them on identical inputs, and visualizing the quality gap between strategies over time.

The secondary objective is to produce a system with genuine software engineering depth — layered architecture, clean module boundaries, testable algorithm modules, the strategy pattern applied meaningfully, and a UI that communicates algorithmic behavior to a non-technical observer.

The tertiary objective is to build a portfolio piece that demonstrates professional-level thinking about systems design, algorithmic analysis, and engineering trade-offs — something a technical interviewer at a logistics technology company would recognize as reflecting real operational concerns.

Target Users

The primary user during the course demonstration is the operator — a person who initializes the simulation, selects a scenario, steps through or runs the simulation, injects manual disruption events, and reads the end-of-shift report. The operator does not need to understand the algorithms to use the system, but the system must expose enough algorithmic detail (DP tables, insertion cost calculations, approximation ratios) that a technical evaluator can verify correctness.

Operational Scenario

A shift begins. Six couriers depart from a central depot with pre-assigned orders, their initial routes computed by a batch DP optimization. At tick 12, a new high-priority order arrives. Greedy insertion finds the cheapest position across all six routes and inserts it in 40 milliseconds. At tick 27, a road closure eliminates a major arterial. Dijkstra re-runs from the three couriers

whose routes use that segment and produces updated routes in 3 milliseconds. At tick 43, the system detects it has been 8 ticks since the last disruption. It schedules a DP re-optimization for Courier 4, whose route has the largest quality gap versus its greedy shadow. The DP runs within its 200ms budget, finds a reordering that saves 3.2 kilometers, and updates the route. The route quality chart shows the step-down improvement. At tick 71, Courier 2's vehicle breaks down. Bin packing FFD redistributes their 4 remaining orders to Couriers 3 and 5. Greedy insertion places the redistributed orders into optimal positions. The shift ends at tick 100 with the end-of-shift report showing: 48 orders delivered, 91.7% on-time rate, 2.8km saved by DP versus greedy-only shadow, FFD outperformed First Fit by 12% on wasted capacity across the shift.

Educational and Portfolio Value

Algorithmically: the project covers Dijkstra, held-karp DP, greedy approximation, and bin packing with genuine integration — not four separate algorithm demonstrations but four modules that share state and interact in a pipeline. The comparison between exact and approximate solutions with measured approximation ratios satisfies the course's core evaluation criteria.

As a portfolio piece: the project demonstrates event-driven simulation design, the strategy pattern applied to algorithm modules, layered architecture with clean dependency direction, a meaningful comparison framework, and the ability to translate algorithmic theory into a working, explainable system. These are skills that distinguish a software engineer from a programmer.

2. Full System Scope and Boundaries

Committed Scale

These parameters are fixed for the entire project. No section of this plan assumes a larger or smaller scale.

Road network graph: 25–30 nodes, 60–80 directed weighted edges
Active couriers: 4–6 simultaneously
Orders per shift: 30–50 total
Simulation ticks: 100 total
DP stop cap: 10 remaining stops per courier
Brute-force cap: 5 remaining stops per courier
Cargo slots: 4–5 per courier
Event queue depth: Never exceeds 200 events in a run
Distance matrix: 30×30 maximum (900 pre-computed paths)

Technical Constraints

The simulation is single-threaded. All algorithm execution is synchronous within a simulation tick. No background threads, no async execution, no concurrent algorithm calls.

The road network graph is fixed at initialization. Node positions do not change during a simulation. Edge weights change (road congestion events) and edges can be removed (road closure events), but no new nodes or edges are added during a run.

The DP module never runs when a courier has more than 10 remaining stops. This is a hard cap enforced by the event dispatcher before the DP module is called. If a courier exceeds the cap when a quiet-period event fires, the event is logged as "DP deferred" and greedy routing is retained.

Assumptions

The road network is modeled as a simplified urban district, not a real city. Node coordinates are fixed and plausible (reflecting a real grid structure) but not derived from actual geographic data. Travel time between connected nodes is represented as an integer weight in the range 1–20 simulated minutes.

All orders have a known weight and volume at the time they enter the system. There are no partial deliveries and no order splitting across multiple couriers. Each order is assigned to exactly one courier.

All couriers start from a central depot at tick 0. The depot is a special node that does not count as a delivery stop.

Non-Goals

This project does not implement incremental Dijkstra. Full single-source Dijkstra re-runs from affected couriers are used throughout. At the committed graph scale, full re-runs are computationally appropriate and produce guaranteed-correct results. Incremental Dijkstra is a research-level problem that is explicitly excluded.

This project does not implement multi-depot routing, vehicle speed variation, turn-by-turn navigation, or real geographic coordinate mapping.

This project does not connect to any external API, real-time traffic feed, or map service.

This project does not implement user authentication, session management, or multi-user support.

This project does not require a relational database. All state is in-memory during a simulation run. The only persistent output is a JSON report file written at shift end.

3. Detailed Algorithm Design

Algorithm 1 — Dijkstra's Shortest Path

Purpose

Dijkstra maintains the system's knowledge of optimal travel costs between all relevant node pairs. Every other algorithm in the system depends on accurate distance estimates — greedy insertion's insertion cost calculation, DP's route permutation evaluation, and bin packing's courier selection logic all read from the pre-computed distance matrix. When any edge in the graph changes, the distance matrix becomes stale for paths that traverse that edge, and Dijkstra must refresh it before other algorithms can produce correct outputs.

Workflow

At simulation initialization, Dijkstra runs once from every node in the graph and the results are stored in a 30×30 distance matrix where `distMatrix[i][j]` is the minimum travel cost from node i to node j . This pre-computation runs once and costs $O(V \times (V + E) \log V)$ — at the committed scale, this is $30 \times (30 + 70) \times \log(30) \approx 14,700$ operations, completing in under 1ms in Java.

During simulation, Dijkstra is triggered by two event types. On road weight change (congestion), the edge weight is updated in the graph and then Dijkstra re-runs from the source node of the changed edge and all courier current positions that have a pre-computed path through that edge. On road removal (closure), the edge is removed from the graph entirely and Dijkstra re-runs from all courier current positions. After re-computation, the distance matrix rows corresponding to the re-run source nodes are updated. All other matrix entries remain valid.

This selective re-run approach — running Dijkstra from affected sources only, not from all 30 nodes — is not incremental Dijkstra. It is a simple optimization: if a courier's current position is node 7 and the disrupted edge has no path through node 7 to any of that courier's remaining stops, there is no need to re-run from node 7. This selection logic is $O(k \times n)$ — checking each of k couriers against each of n remaining stops — and is correct, not an approximation.

Inputs and Outputs

Input: Graph G (adjacency list with current edge weights)

Source node s

Output: `distMap[v]` = minimum cost from s to all reachable nodes v

`predecessor[v]` = previous node on optimal path to v

The predecessor map enables path reconstruction — not just the cost, but the actual sequence of nodes — which is needed to check whether a courier's route traverses a closed edge.

Complexity

Single-source Dijkstra with a binary heap priority queue: $O((V + E) \log V)$. At committed scale: $O((30 + 70) \times \log 30) \approx 500$ operations per run. With at most 6 couriers re-running after a disruption: 3,000 operations. Under 0.1ms in Java.

Correctness Strategy

Three unit tests with hand-calculated expected outputs on a known 8-node graph. Verify that: the algorithm produces the correct shortest distance for every source-destination pair, the predecessor map enables correct path reconstruction, and the result does not change if the algorithm is run twice on the same graph (idempotency).

Interaction with Other Modules

Dijkstra always executes before any other algorithm in the same tick. The event dispatcher enforces this order. After Dijkstra updates the distance matrix, greedy insertion and DP both read from the updated matrix. This sequencing guarantee is documented in the event dispatcher and enforced by the single-threaded execution model — there is no concurrency to manage.

Discussion: Full Re-Run vs. Incremental vs. Localized

Full re-run from all sources: $O(V \times (V + E) \log V)$ per event. At committed scale, this is 14,700 operations — still under 2ms. Correct but wasteful.

Localized re-run from affected sources: $O(k \times (V + E) \log V)$ where k is affected couriers. At committed scale with 6 couriers: 3,000 operations. Correct, fast, implemented in this project.

Incremental Dijkstra (D^* , LPA^*): avoids re-processing unaffected nodes. Requires complex data structure maintenance. Research-level complexity. Incorrect outputs possible if not implemented exactly. Explicitly excluded at this project's scale — the performance gain is not worth the implementation risk.

Algorithm 2 — Dynamic Programming Route Re-Optimization

Purpose

The DP re-optimizer addresses the accumulated suboptimality in courier routes that have been built up through repeated greedy insertions. Each greedy insertion is locally optimal — it finds the cheapest position for one new order given the current route — but does not globally re-evaluate the entire remaining sequence. Over many insertions, the route diverges from optimal. The DP module periodically re-evaluates the full permutation of remaining stops and finds the globally optimal ordering, improving the route in ways that greedy insertion cannot.

Held-Karp Style Dynamic Programming

The state definition is: $dp[S][v]$ = minimum total distance to visit all delivery stops in subset S , ending at stop v , starting from the courier's current position.

The transition is: for each stop u not yet in S , $dp[S \cup \{u\}][u] = \min(dp[S][v] + distMatrix[v][u])$ for all v in S .

The base case is: $dp[\{s\}][s] = distMatrix[current_position][s]$ for each stop s in the remaining set.

The solution is: $\min \text{ over all } v \text{ in full_set of } (dp[full_set][v] + distMatrix[v][depot])$ — the minimum-cost completion of all stops, ending at the depot.

Backtracking: the optimal ordering is recovered by recording, for each $dp[S][v]$, which previous stop v' produced the minimum. Follow the backtrack pointers from the solution state to the base cases.

State Space and Stop Cap

The state space size is $O(2^n \times n)$ where n is the number of remaining stops. At the committed cap of 10 stops: $2^{10} \times 10 = 10,240$ states. This fills a two-dimensional array of 10,240 floats — approximately 40KB in Java. Runs in under 100ms in Java at $n=10$. At $n=12$: 49,152 states, still under 200ms. At $n=15$: 491,520 states, may exceed the 200ms budget. At $n=20$: 20,971,520 states — unacceptable. The hard cap at 10 stops is therefore a correctness constraint, not a performance shortcut.

Time Budget and Anytime Behavior

The DP module accepts a time budget parameter (default: 200ms wall-clock time). The module checks elapsed time at the completion of each row of the DP table (each value of $|S|$). If elapsed time exceeds the budget when starting a new row, the module stops and returns the best complete solution found so far. A "complete solution" is a state where $dp[S][v]$ is fully populated for the current $|S|$ — partial row computation is discarded.

At the committed stop cap of 10, the DP almost always completes within 200ms without hitting the budget. The budget mechanism exists as a safety valve for edge cases — a courier that somehow has 11 stops due to redistribution, or hardware that is slower than expected. Logging must distinguish between "DP completed" and "DP budget exceeded, returning best-so-far" — these are different operational states.

Correctness Verification

For any courier with 5 or fewer remaining stops, the brute-force verifier runs in parallel with DP. The brute-force enumerates all $(n-1)!$ orderings (for $n=5$: 24 orderings), computes total distance for each, and returns the minimum. The DP solution and brute-force solution must agree exactly. Any disagreement is a bug in the DP implementation. This verification runs in the testing phase and in the simulation's verbose mode — not in production demo mode.

Quiet-Period Trigger

DP fires when the QuietPeriodMonitor detects that the simulation has processed no disruption events for N consecutive ticks (default $N=5$). When this condition is met, the dispatcher fires a QUIET_PERIOD event for each eligible courier — any courier with 3 or more remaining stops and 10 or fewer remaining stops. If multiple couriers are eligible, they are prioritized by estimated improvement gap (the difference between current route distance and the lower-bound estimate from the distance matrix alone).

Interaction with Shadow Route

The shadow route is the critical companion to the DP module. It is a parallel copy of each courier's route that receives all greedy insertions but never receives DP improvements. The quality gap at any simulation tick is $(\text{shadow_route_distance} - \text{dp_route_distance}) / \text{shadow_route_distance}$. This gap is the only metric that measures genuine DP improvement versus natural route shortening from delivery completions. The shadow route costs one additional data structure update per greedy insertion event — essentially free — and enables the most important visualization in the system.

Complexity

DP: $O(2^n \times n^2)$ for the full table. At $n=10$: 102,400 operations. Brute-force verifier: $O(n!)$ at $n=5$: 120 permutations. Shadow route update: $O(1)$ per insertion.

Algorithm 3 — Greedy Cheapest Insertion

Purpose

Greedy insertion is the system's real-time response mechanism. When a new order arrives, the system cannot wait 200ms for DP to compute an optimal position — the operator needs to see the order assigned immediately, the courier's route updated, and the system ready for the next event. Greedy insertion evaluates all valid insertion positions across all active couriers and selects the globally minimum detour cost, producing a valid route assignment in under 5ms at the committed scale.

Workflow

For a new order O with delivery location node d , and a courier C with current route $[c_0, c_1, c_2, \dots, c_n]$ (where c_0 is the current position and c_n is the last remaining stop before depot return):

For each insertion position i from 1 to $n+1$:

$$\begin{aligned} \text{insertion_cost}(i) = & \text{distMatrix}[c_{i-1}][d] \\ & + \text{distMatrix}[d][c_i] \\ & - \text{distMatrix}[c_{i-1}][c_i] \end{aligned}$$

This is the detour cost: how much extra distance is added by detouring through d between stop $i-1$ and stop i , compared to traveling directly.

The algorithm evaluates all positions across all k couriers, finds the globally minimum insertion cost, and places the order there. It also checks that the receiving courier has sufficient cargo capacity before committing the insertion. If the minimum-cost courier is over capacity, the algorithm selects the next-minimum-cost eligible courier.

Complexity

$O(k \times m)$ insertion position evaluations, where k is the number of couriers and m is the average number of remaining stops. At committed scale: 6 couriers $\times$ 8 stops average = 48 evaluations. Each evaluation is a constant number of array lookups in the distance matrix. Total: under 0.1ms in Java. The guide's stated $O(n)$ complexity is incorrect — the correct complexity is $O(k \times m)$, which at this scale is effectively equivalent but must be stated accurately in the comparison report.

Priority Weighting

For priority escalation events, the insertion cost formula includes a priority weight term:

$$\text{weighted_cost}(i) = \text{insertion_cost}(i) + \text{priority_penalty} \times \text{estimated_delay}(i)$$

Where $\text{estimated_delay}(i)$ is the estimated additional simulated minutes before delivery if the order is inserted at position i . This biases the selection toward earlier positions for high-priority orders, accepting higher route cost in exchange for faster delivery.

Trade-offs

Greedy insertion is locally optimal — it makes the best decision given the current state — but accumulates suboptimality over multiple insertions because each insertion treats the current route as fixed. The classic failure case is the cluster problem: when orders arrive alternating between two geographic clusters, greedy insertion crosses between clusters repeatedly rather than grouping deliveries within each cluster. This failure case is deliberately engineered into the demonstration graph to make the DP improvement visible.

Algorithm 4 — Bin Packing Cargo Re-Assessment

Purpose

Bin packing in this system serves two functions. First, it validates cargo feasibility after every order insertion — ensuring no courier's vehicle is loaded beyond its weight or volume capacity. Second, it manages order redistribution when a vehicle breakdown creates an unassigned order pool that must be assigned to remaining couriers.

Three Strategies

First Fit (FF): For each order to be placed, scan couriers in their default order and assign to the first courier with sufficient remaining capacity. $O(k \times n)$ where k is couriers and n is orders. Simple, fast, suboptimal. Best case: works well when all orders are similar size. Worst case: fills first couriers to near-capacity and leaves later couriers underutilized.

Best Fit (BF): For each order, find the courier whose remaining capacity is closest to (but not less than) the order size. This minimizes wasted space per assignment. $O(k \times n)$ with a slightly higher constant. Produces tighter packing than FF, particularly when cargo sizes vary.

First Fit Decreasing (FFD): Sort all orders by size descending, then apply First Fit. This is the theoretically strongest strategy — the FFD approximation guarantee states the result is at most $(11/9)\text{OPT} + 6/9$ bins. In practice, it produces near-optimal results on bimodal size distributions (the distribution used in this project). The pre-sort costs $O(n \log n)$, which dominates the overall complexity.

Operational Impact

For the comparison report, all three strategies run on the same redistribution event (vehicle breakdown) and their results are compared: couriers used, wasted capacity percentage, and orders unassigned (if any). The approximation ratio for each strategy is computed as $(\text{strategy_result} / \text{theoretical_lower_bound})$ where the theoretical lower bound is $\text{ceil}(\text{total_cargo_size} / \text{max_cargo_capacity})$.

Cargo Size Distribution

Orders use a bimodal size distribution: 30% of orders are large (occupying 60–70% of a cargo slot) and 70% are small (occupying 15–20% of a cargo slot). This distribution is adversarial for First Fit — it produces measurable differences between strategies that would not appear with uniform-size orders.

Interaction with Greedy Insertion

Bin packing re-assessment runs immediately after greedy insertion in the event handler chain. It does not produce a route change — it validates capacity and either confirms the insertion or triggers redistribution. Redistribution calls greedy insertion to place redistributed orders into the receiving couriers' routes. This means bin packing and greedy insertion can form a call chain: greedy → bin packing → redistribution trigger → greedy (for each redistributed order).

4. Event-Driven Simulation Design

Simulation Engine Overview

The simulation engine is the orchestration layer that drives the entire system. It maintains the simulation clock, processes events from the priority queue, dispatches algorithm calls, applies mutations to state, and emits state snapshots to the API layer. It contains no algorithm logic. It is a coordinator, not a solver.

SimClock

The SimClock is a simple integer counter that advances by one per tick. It does not use wall-clock time internally. The relationship between simulation ticks and wall-clock time is controlled by the UI's playback speed slider, which determines how many milliseconds the frontend waits

between requesting the next tick. The simulation engine itself is agnostic to playback speed — it processes one tick per call to `step()`.

Event Queue

The event queue is a priority queue ordered by simulated tick. Each event has a `scheduledTick` field. Events scheduled for the same tick are processed in priority order — disruption events before quiet-period events, within disruptions, road events before order events (because road events update the distance matrix that order event algorithms depend on).

Event priority order within a tick:

1. ROAD_REMOVAL (highest — affects all subsequent events)
2. ROAD_WEIGHT_CHANGE
3. VEHICLE_BREAKDOWN (order redistribution before new insertions)
4. NEW_ORDER
5. PRIORITY_ESCALATION
6. QUIET_PERIOD (lowest — only if no other events in this tick)

Event Types and Handlers

NEW_ORDER

Preconditions: order not already assigned

Algorithm chain:

1. GreedyInsertionModule.insert(routes, order, distMatrix)
2. BinPackingModule.validateCapacity(courier, order)
3. If violation: BinPackingModule.redistribute(order, routes)
4. ShadowRouteRegistry.applyInsertionToShadow(order, insertionResult)

State mutations: route registry, cargo registry, order registry

Quiet-period timer: RESET

ROAD_WEIGHT_CHANGE

Preconditions: edge exists in graph

Algorithm chain:

1. GraphStore.updateEdgeWeight(edge, newWeight)
2. DijkstraModule.recompute(graph, affectedSources)
3. DistanceMatrix.updateRows(affectedSources, newDistances)

State mutations: graph store, distance matrix

Quiet-period timer: RESET

ROAD_REMOVAL

Preconditions: edge exists in graph

Algorithm chain:

1. GraphStore.removeEdge(edge)
2. Identify couriers whose routes traverse removed edge
3. DijkstraModule.recompute(graph, affectedCouriers.currentPositions)
4. DistanceMatrix.updateRows(affectedPositions, newDistances)
5. For each affected courier: recompute route segments

State mutations: graph store, distance matrix, route registry

Quiet-period timer: RESET

VEHICLE_BREAKDOWN

Preconditions: courier is active

Algorithm chain:

1. CourierRegistry.deactivate(courierId)
2. Extract undelivered orders from courier manifest
3. BinPackingModule.redistribute(orders, activeRoutes, FFD)
4. For each redistributed order:
GreedyInsertionModule.insert(routes, order, distMatrix)

State mutations: courier registry, route registry, cargo registry

Quiet-period timer: RESET

PRIORITY_ESCALATION

Preconditions: order is assigned

Algorithm chain:

1. GreedyInsertionModule.reinsert(routes, order, distMatrix,
priorityWeight=HIGH)

State mutations: route registry

Quiet-period timer: RESET

QUIET_PERIOD

Preconditions: no event in last N ticks

Algorithm chain:

1. For each eligible courier (3–10 remaining stops):
 - a. DPReoptimiserModule.optimise(courier, distMatrix, budgetMs)
 - b. If improvement > threshold: RouteRegistry.update(courier, newRoute)
 - c. MetricsStore.logDPImprovement(courier, delta)

State mutations: route registry

Quiet-period timer: NOT RESET (quiet period continues)

Quiet Period Detection

The QuietPeriodMonitor maintains a `ticksSinceLastDisruptionEvent` counter. Every time a disruption event (any event except QUIET_PERIOD) is processed, this counter resets to 0. Every tick that passes without a disruption event increments the counter. When the counter reaches the configured threshold N (default: 5), the monitor schedules a QUIET_PERIOD event for the next tick. If the QUIET_PERIOD event is processed and the counter is still at or above N, the monitor schedules another QUIET_PERIOD for the following tick, allowing continuous DP optimization during extended quiet windows.

Mutation Flow

No algorithm mutates state directly. Every algorithm function returns a `Mutation` object describing what should change. The simulation engine's `MutationApplier` validates and commits each mutation after verifying consistency constraints. This design ensures that a buggy algorithm cannot corrupt the simulation state — the applier is the single gate through which all mutations pass.

Mutation types:

RouteInsertionMutation: courier, position, order

RouteReorderMutation: courier, newStopSequence

EdgeWeightMutation: edge, newWeight

EdgeRemovalMutation: edge

CourierDeactivationMutation: courierId, undeliveredOrders

5. Functional Requirements

Core Simulation Requirements

FR-SIM-01: The system must support a discrete-event simulation with a configurable tick count (default 100) and step-by-step or auto-run playback modes.

FR-SIM-02: The system must initialize from a predefined scenario file specifying graph topology, courier configurations, order schedules, and disruption event schedules.

FR-SIM-03: The system must support at least two pre-scripted demonstration scenarios launchable from the UI with one click.

FR-SIM-04: The system must support simulation reset to initial state without application restart.

FR-SIM-05: The system must support manual disruption injection by the operator during simulation playback.

Algorithmic Requirements

FR-ALG-01: Dijkstra must run as a full single-source computation from each affected source after every graph mutation. Distance matrix must be refreshed before any routing algorithm executes in the same tick.

FR-ALG-02: Greedy insertion must evaluate all valid insertion positions across all active couriers using the current distance matrix and return the global minimum-cost valid insertion.

FR-ALG-03: DP re-optimization must implement the held-karp state formulation $dp[S][v]$, run within a configurable time budget, and return the best complete solution found. Must not run for couriers with more than 10 remaining stops.

FR-ALG-04: Brute-force verification must run for all DP calls where the courier has 5 or fewer remaining stops and must confirm agreement with the DP result.

FR-ALG-05: Bin packing must implement and execute all three strategies (FF, BF, FFD) on every redistribution event, logging the result of each for comparison.

FR-ALG-06: All algorithm modules must conform to the `AlgorithmModule` interface and return a `Mutation` plus a `MetricsRecord`.

Route and State Management Requirements

FR-STATE-01: The system must maintain a shadow route for each courier from the first insertion event. The shadow route receives all greedy insertions and delivery completions but never receives DP improvements.

FR-STATE-02: Route quality must be measured as `distance_per_remaining_stop` (total remaining distance / count of remaining stops) for both the active route and the shadow route.

FR-STATE-03: The distance matrix must be a single shared object. Only the Dijkstra module writes to it. All other modules read from it.

UI and Visualization Requirements

FR-UI-01: The live delivery map must display all graph nodes, all edges (colored by current weight), all active courier routes as colored polylines, and all courier positions as labeled markers. Updates after every simulation tick.

FR-UI-02: The event log must display all processed events with: simulated tick, event type, affected entities, algorithm used, execution time in milliseconds, and route cost delta.

FR-UI-03: The route quality chart must display `distance_per_remaining_stop` for each courier's active route and shadow route as separate time-series lines.

FR-UI-04: The greedy vs. DP comparison panel must display: cumulative distance saved by DP per courier, total fleet distance saved, number of DP cycles completed, average DP improvement per cycle, and average DP execution time.

FR-UI-05: The bin packing diagram must display a slot-fill bar for each courier showing loaded cargo sizes with color coding by order type, with wasted capacity visually indicated.

FR-UI-06: The DP table viewer must display the `dp[S][v]` table for a selected courier on demand, with the optimal path highlighted.

FR-UI-07: The simulation control panel must expose: Step, Run, Pause, Reset, Speed slider (1–10x), Scenario selector, and Manual event injection form.

Reporting Requirements

FR-RPT-01: The end-of-shift report must be generated at simulation completion and include all metrics defined in Section 16.

FR-RPT-02: The report must be exportable as a formatted JSON file.

FR-RPT-03: The report panel must be rendered in the UI as a structured dashboard, not a raw JSON display.

6. Non-Functional Requirements

Performance

All algorithms must complete within a single simulation tick without perceptible delay at committed scale. Measured performance targets:

Dijkstra (per re-run from one source):	< 2ms
Greedy insertion (6 couriers × 10 stops):	< 5ms
DP re-optimization (10 stops, full table):	< 200ms
Bin packing (all 3 strategies, 10 orders):	< 2ms
Full tick processing (all events in one tick):	< 250ms
UI render after tick update:	< 100ms (target 60fps)
SSE state snapshot serialization:	< 10ms

Correctness

DP results must match brute-force for all inputs with $n \leq 5$. Dijkstra results must match hand-calculated values for all test graph cases. Bin packing results must never produce an assignment where any courier's total cargo exceeds their capacity. Greedy insertion must never place an order in a position that increases route cost above the maximum alternative valid insertion.

Reproducibility

Given identical initial state (same graph, same courier config, same scenario file), the simulation must produce identical output on every run. The event queue must be deterministic. Any randomness used in scenario generation must be seeded with a fixed seed. This requirement enables scripted demonstration scenarios to work reliably.

Modularity

Any single algorithm module must be replaceable without modifying the simulation engine, the state layer, or the UI. This is validated by the strategy pattern — the engine calls algorithms through the `AlgorithmModule` interface, with no direct dependency on concrete implementations.

Observability

Every algorithm call must produce a log entry. Every state mutation must produce a log entry. The event log must be the complete and sufficient audit trail for any simulation run — given only the initial state and the event log, it must be possible to reconstruct the full simulation output.

Demonstrability

The quality gap between greedy-only and DP-optimized routes must be visually distinct on the route quality chart when using the adversarial demonstration scenario. The gap must be at least 10% in distance-per-remaining-stop by tick 60 of the demonstration scenario. If it is not, the scenario must be adjusted until it is — the demonstration scenario is a designed artifact, not a random input.

Scalability Boundaries

The system is not designed to scale beyond the committed parameters. Doubling the graph size to 60 nodes would require re-evaluating the Dijkstra strategy. Doubling the DP stop cap to 20 would make DP computationally infeasible. The committed scale is the correct operating envelope and any pressure to exceed it should be resisted.

7. Full Project Architecture

Architecture Style: Modular Monolith

A modular monolith is the correct architecture for this project. All algorithms share the same mutable state — the distance matrix, the route registry, the cargo registry. In a microservices architecture, sharing this state would require serializing it on every inter-service call, adding network overhead to every algorithm invocation. At the committed scale, this overhead would dominate the algorithm execution time. Worse, it would introduce consistency risks — a microservice holding a stale copy of the distance matrix would produce incorrect routes. The single-process shared-memory model eliminates this entire class of problems.

Within the modular monolith, modules are separated by strict layer boundaries enforced at the package level. The algorithm package has no dependency on the simulation package. The simulation package has no dependency on the API package. The API package has no dependency on the algorithm package (it only knows about state snapshots and mutations). This dependency structure is enforced by code review and validated by unit tests that import only the modules they should depend on.

Full Layered Architecture

PRESENTATION LAYER (React SPA)

Purpose: Render simulation state. Accept operator input.

Dependencies: API layer only (via SSE and REST)

No business logic. No algorithm logic.

↕ REST (control commands) + SSE (state updates)

API LAYER (Spring Boot Controllers + SSE)

Purpose: Translate HTTP requests to simulation engine calls.

Serialize state snapshots to JSON.

Stream state updates via SSE.

Dependencies: Simulation Engine, State Layer (read-only)
No algorithm logic. No direct state mutation.

↓ Method calls

SIMULATION ENGINE (SimClock, EventQueue, EventDispatcher, MutationApplier)

Purpose: Drive the simulation. Orchestrate algorithm calls.

Apply mutations. Emit state snapshots.

Dependencies: Algorithm Layer (via interface), State Layer

No algorithm logic (no shortest path, no DP computation here).

No UI logic. No HTTP logic.

↓ Pure function calls (in, out)

ALGORITHM LAYER (Dijkstra, GreedyInsertion, DPReoptimiser, BinPacking)

Purpose: Compute solutions to optimization sub-problems.

Dependencies: State Layer (read-only during computation)

Pure functions. No side effects. No UI. No simulation state mutation.

↓ Read-only access

STATE LAYER (GraphStore, RouteRegistry, CargoRegistry, DistanceMatrix,
OrderRegistry, EventLog, MetricsStore, ShadowRouteRegistry)

Purpose: Authoritative source of simulation state.

Dependencies: None (no upward dependencies)

Mutated only by MutationApplier. Read by all other layers.

Data Flow

Operator action → REST endpoint → SimulationEngine.step() or injectEvent() →

EventDispatcher → algorithm module call → Mutation returned →

MutationApplier.apply(mutation) → State updated → StateEmitter.snapshot() → SSE push →

React component re-render.

Dependency Direction Rule

Dependencies flow downward only. The presentation layer depends on the API layer. The API layer depends on the simulation engine and state layer. The simulation engine depends on the algorithm layer and state layer. The algorithm layer depends on the state layer (read-only). The state layer depends on nothing. No upward dependencies. No circular dependencies.

8. Complete System Design

Design Principles

Single Responsibility Principle: Every class has exactly one reason to change.

DijkstraModule changes only if the shortest-path algorithm changes. EventDispatcher

changes only if the event-to-algorithm mapping changes. `RouteRegistry` changes only if the data model for courier routes changes.

Open/Closed Principle: The algorithm layer is open for extension (new algorithms can be added by implementing `AlgorithmModule`) and closed for modification (adding a new algorithm does not change any existing algorithm or the simulation engine).

Liskov Substitution Principle: Any `AlgorithmModule` implementation can replace any other in the dispatcher without breaking the simulation engine. This is the property that makes the strategy pattern meaningful.

Interface Segregation Principle: Algorithm modules depend only on the parts of the state layer they actually need. `DijkstraModule` depends on `GraphStore` and `DistanceMatrix`. It has no knowledge of `RouteRegistry` or `CargoRegistry`.

Dependency Inversion Principle: The simulation engine depends on the `AlgorithmModule` interface abstraction, not on `DijkstraModule`, `GreedyInsertionModule`, or `DPreoptimiserModule` directly. Concrete implementations are injected at construction time.

Strategy Pattern — Algorithm Modules

```
java
public interface AlgorithmModule<I extends AlgorithmInput,
    O extends AlgorithmOutput> {
    O solve(I input);
    String algorithmName();
    AlgorithmComplexity reportedComplexity();
}

public interface AlgorithmInput {
    SystemStateSnapshot getStateSnapshot();
    Event getTriggeringEvent();
}

public interface AlgorithmOutput {
    Mutation getMutation();
    MetricsRecord getMetrics();
}
```

Concrete implementations:

```
java
public class DijkstraModule
    implements AlgorithmModule<DijkstraInput, DijkstraOutput> { ... }

public class GreedyInsertionModule
    implements AlgorithmModule<InsertionInput, InsertionOutput> { ... }

public class DPreoptimiserModule
    implements AlgorithmModule<DPInput, DPOutput> { ... }
```

```
public class BinPackingModule
    implements AlgorithmModule<BinPackingInput, BinPackingOutput> { ... }
```

The event dispatcher holds references to `AlgorithmModule` interfaces, not concrete classes. Concrete implementations are registered in a configuration class and injected via Spring's dependency injection.

State Immutability and Snapshot Pattern

The state layer holds mutable objects during simulation execution. However, when the simulation engine requests a state snapshot for the API layer to serialize and push to the UI, the snapshot is an immutable deep copy of the current state. Algorithm modules receive a `SystemStateSnapshot` — an immutable view — not the live mutable state. This prevents algorithms from accidentally mutating state during computation.

```
java
public class SystemStateSnapshot {
    private final Map<String, CourierStateSnapshot> couriers; // immutable
    private final GraphSnapshot graph; // immutable
    private final double[][] distanceMatrix; // copied
    private final List<OrderSnapshot> orders; // immutable
    private final List<EventLogEntry> recentEvents; // immutable

    // only getters, no setters
}
```

Clean Code Patterns

Command Pattern for Mutations: Each mutation is a command object with an `apply(StateLayer state)` method. The `MutationApplier` calls `mutation.apply(state)`. New mutation types are added by implementing the `Mutation` interface, not by adding cases to a switch statement.

Builder Pattern for Events: Events have multiple optional fields. Event construction uses the builder pattern to ensure valid, complete event objects without constructor overload explosions.

```
java
Event event = Event.builder()
    .type(EventType.NEW_ORDER)
    .scheduledTick(42)
    .priority(EventPriority.NORMAL)
    .payload(new NewOrderPayload(order))
    .build();
```

Factory Method for Algorithm Modules: Algorithm module instances are created by a factory that reads configuration and returns the appropriate implementation. This decouples the dispatcher from construction logic.

Null Object Pattern for Optional Metrics: When an algorithm does not produce a meaningful metric (for example, Dijkstra has no "approximation ratio"), it returns a `NullMetricsRecord` that implements `MetricsRecord` but returns zero or N/A for all fields. This avoids null checks throughout the metrics logging code.

Value Objects for Domain Concepts: Core domain concepts — `Distance`, `SimulatedTick`, `CourierId`, `NodeId` — are represented as value objects, not raw primitives. This prevents a class of bugs where a courier ID is passed as a distance or a tick count is treated as a node ID.

```
java
public record CourierId(String value) {}
public record NodeId(int value) {}
public record SimulatedTick(int value) implements Comparable<SimulatedTick> { ... }
public record Distance(double kilometers) {}
```

Guard Clauses over Nested Conditionals: Algorithm implementations use guard clauses at the top of methods rather than deeply nested if-else trees. This keeps the happy path at the leftmost indentation level and makes precondition violations immediately visible.

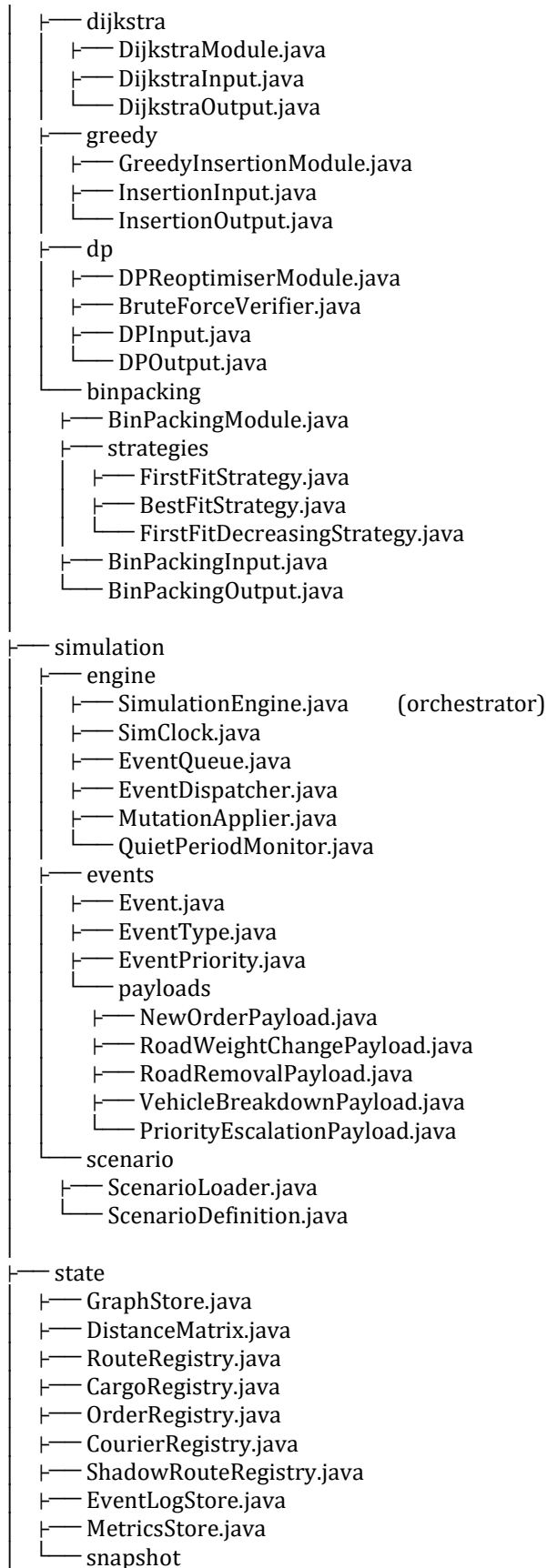
```
java
public DPOutput solve(DPInput input) {
    if (input.getRemainingStops().size() > MAX_DP_STOPS) {
        return DPOutput.deferred("stop cap exceeded");
    }
    if (input.getRemainingStops().size() < MIN_DP_STOPS) {
        return DPOutput.deferred("insufficient stops");
    }
    // main DP computation
}
```

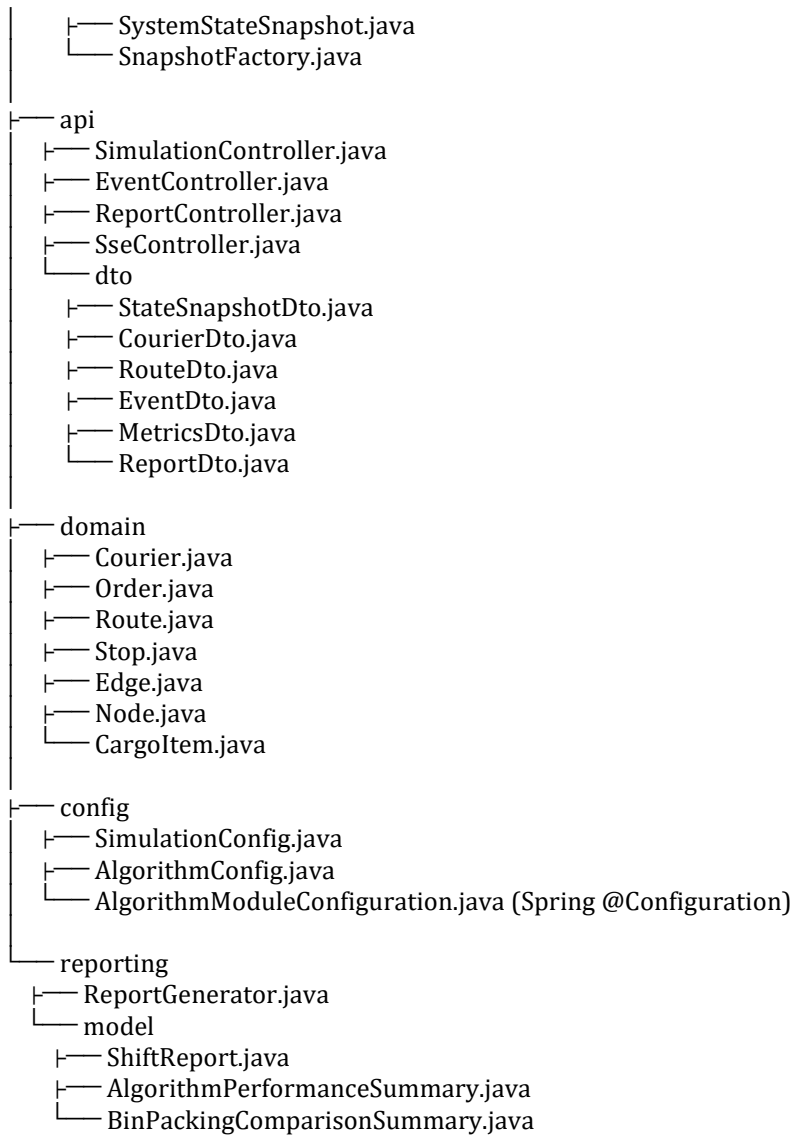
Meaningful Return Types over Exceptions for Expected Conditions: Algorithms return typed result objects that carry success/failure/deferred status rather than throwing exceptions for expected operational conditions (like the DP stop cap being exceeded). Exceptions are reserved for programming errors and truly unexpected failures.

9. Backend Design

Package Structure

```
com.deliveryoptimizer
├── algorithm
│   └── api
│       ├── AlgorithmModule.java    (interface)
│       ├── AlgorithmInput.java     (interface)
│       ├── AlgorithmOutput.java     (interface)
│       ├── Mutation.java           (interface)
│       └── MetricsRecord.java       (interface)
```





REST API Endpoints

POST /api/simulation/init
Body: ScenarioDefinition (scenario name or custom config)
Response: SimulationStatusDto (tick 0, initial state)

POST /api/simulation/step
Body: empty
Response: StateSnapshotDto (state after tick advance)

POST /api/simulation/run
Body: RunRequest { ticks: int, speedMultiplier: float }
Response: 200 OK (state streamed via SSE during run)

POST /api/simulation/pause
Response: StateSnapshotDto (current state)

POST /api/simulation/reset
Response: StateSnapshotDto (initial state restored)

POST /api/events/inject
Body: EventDto (type, payload, targetTick)
Response: EventAcknowledgement

GET /api/simulation/state
Response: StateSnapshotDto (current state, for reconnection)

GET /api/simulation/stream
Response: text/event-stream (SSE)
Events: stateUpdate, simulationComplete, error

GET /api/reports/shift
Response: ShiftReportDto (final report)

GET /api/reports/shift/export
Response: application/json file download

SSE Design

The `SseController` maintains a list of active `SseEmitter` instances. After each simulation tick, the `SimulationEngine` calls `StateEmitter.emit(snapshot)`, which serializes the snapshot to JSON and sends it to all active emitters. Emitters that have timed out or disconnected are removed from the list on send failure.

The SSE stream sends two event types: `stateUpdate` (after each tick) and `simulationComplete` (when tick 100 is reached, includes the final report payload). The frontend's `EventSource` handles both event types with separate listeners.

Error Handling

Algorithm modules return typed result objects that include error states — they do not throw exceptions for expected conditions. The simulation engine handles error-state results by logging them and retaining the previous state (the mutation is not applied). A failed algorithm call never corrupts simulation state.

Spring's `@ControllerAdvice` handles unexpected exceptions at the API layer, returning structured error responses without stack traces. Internal errors are logged with full stack traces to the application log but not exposed to the frontend.

Testing Strategy

Unit tests use JUnit 5. Each algorithm module has a dedicated test class with:

- Correctness tests on small known-answer inputs
- Edge case tests (empty route, single-stop route, maximum stops)
- Performance tests that assert execution time bounds using `Assertions.assertTimeout`

Integration tests use `@SpringBootTest` with a full application context. They run pre-scripted scenarios and assert on the final state — specific couriers on specific routes, specific metrics values. These tests are the regression protection for the simulation engine's event ordering logic.

10. Frontend Design

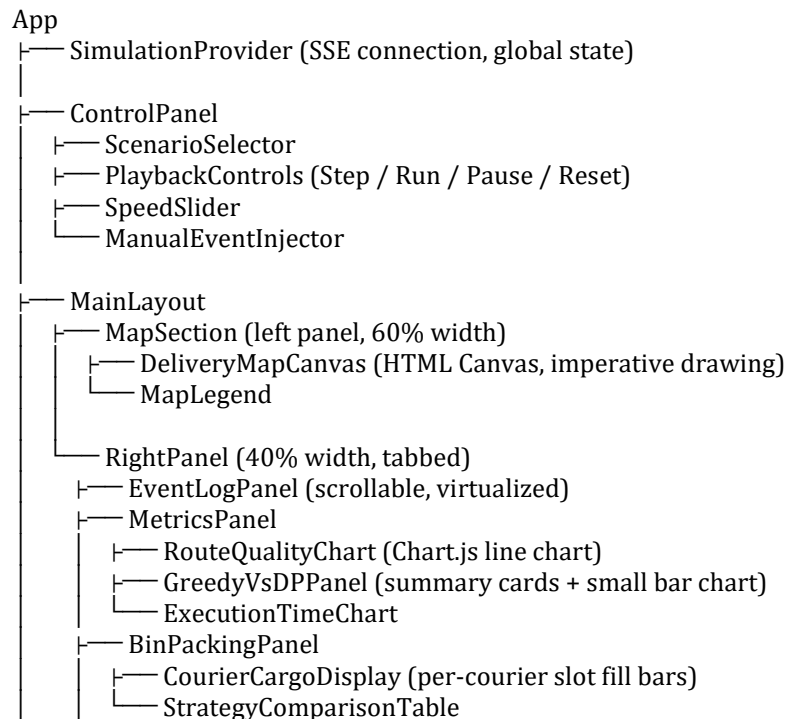
Architecture Justification

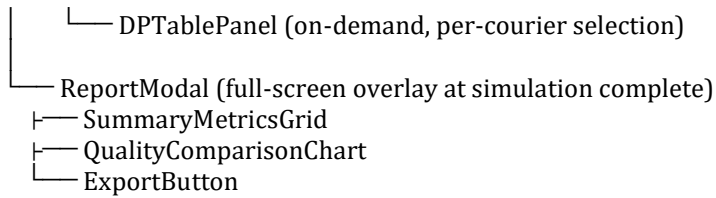
React with Vite and Tailwind CSS is the correct choice. The UI is fundamentally a state display problem — every simulation tick produces a new state snapshot, and every component needs to re-render based on that snapshot. React's component model and unidirectional data flow map exactly to this pattern. Vite's development server with Hot Module Replacement makes frontend iteration fast without requiring a full rebuild cycle. Tailwind's utility classes make layout and styling fast to write without a custom stylesheet.

Next.js is unnecessary — there is no server-side rendering requirement, no SEO requirement, and no file-based routing. The entire UI is a single page. Next.js would add compilation complexity with no benefit.

Vue and Angular are acceptable alternatives but offer no advantage over React for this specific problem, and React aligns with the team's existing experience.

Component Hierarchy





Map Canvas Rendering

The map is rendered on an HTML Canvas element managed by a React ref. On each state snapshot received from the backend, `useEffect` triggers a full canvas redraw. The draw function follows this order:

1. Clear canvas
2. Draw all edges (colored by weight: green → yellow → red as weight increases)
3. Draw MST or route path polylines (colored by courier ID, using a fixed color palette)
4. Draw all node circles (filled, with node ID labels)
5. Draw courier position markers (colored dots, positioned at current node)
6. Draw order completion markers (checkmarks at delivered stop nodes)

Courier positions are not animated between stops — they jump to the next stop on each tick. This is correct and intentional: the map is a state display, not an animation. Smooth movement animation would require interpolation logic that costs implementation time with zero algorithmic value.

State Management

The `SimulationProvider` component holds the current state snapshot in a `useState` hook and manages the SSE connection in a `useEffect` hook. All child components receive state via React context — no prop drilling beyond two levels. No external state management library (Redux, Zustand) is needed — the state shape is simple and the update pattern is straightforward: receive SSE event, parse JSON, call `setState`.

javascript

```
const SimulationProvider = ({ children }) => {
  const [state, setState] = useState(null);
  const [connected, setConnected] = useState(false);

  useEffect(() => {
    const eventSource = new EventSource('/api/simulation/stream');

    eventSource.addEventListener('stateUpdate', (e) => {
      setState(JSON.parse(e.data));
    });

    eventSource.addEventListener('simulationComplete', (e) => {
      setState(prev => ({ ...prev, report: JSON.parse(e.data) }));
    });
  });
};
```

```

    eventSource.onopen = () => setConnected(true);
    eventSource.onerror = () => setConnected(false);

    return () => eventSource.close();
  }, []);

  return (
    <SimulationContext.Provider value={{ state, connected }}>
      {children}
    </SimulationContext.Provider>
  );
};

```

Chart Rendering

Chart.js charts are managed via `useRef` for the chart instance and `useEffect` for data updates. On each new state snapshot, the effect pushes new data points to the chart dataset and calls `chart.update('none')` — the 'none' animation mode updates data without re-rendering the chart animation, which is correct for real-time streaming data.

11. Storage and Persistence Design

In-Memory State Model

All simulation state lives in Java heap memory for the duration of a simulation run. The state layer objects — `GraphStore`, `RouteRegistry`, `DistanceMatrix`, `CargoRegistry`, `OrderRegistry`, `EventLogStore`, `MetricsStore`, `ShadowRouteRegistry` — are Spring-managed beans with `@Scope("prototype")` or managed by the `SimulationEngine` directly, allowing them to be reset cleanly on `reset()` without application restart.

The complete state at any tick is fully representable as a Java object graph with no circular references (the snapshot factory ensures this). This means serialization to JSON is straightforward and reliable.

JSON Export Strategy

At simulation end, the `ReportGenerator` produces a `ShiftReport` object containing all metrics defined in Section 16. Spring's Jackson serializer converts this to a formatted JSON file. The file is written to a configured output directory (default: `./reports/`) with a filename including the scenario name and ISO timestamp.

The JSON report structure is the canonical source of truth for the end-of-shift report panel. The frontend's report panel renders from the same JSON that can be exported, ensuring the displayed report and the exported file are identical.

Reproducibility

Because all state is in-memory and the simulation is deterministic, the same scenario produces the same output on every run. If the scenario file is saved alongside the report JSON, the simulation can be re-run at any time to reproduce the exact same output. This makes the project's results verifiable — a grader can run the scenario, compare the report, and confirm consistency.

Optional Persistence Extension

If persistence across application restarts is desired (for example, to save a simulation midpoint and resume later), the `SystemStateSnapshot` can be serialized to JSON and written to a file at any tick. On application restart, loading this file and restoring the state layer objects from it would resume the simulation from the saved point. This is a one-afternoon extension that does not require a database.

12. Communication Architecture

SSE vs. WebSockets

SSE (Server-Sent Events) is the correct choice for this project. The communication pattern is unidirectional: the server pushes state updates to the client after each simulation tick. The client sends control commands (step, run, reset, inject event) via standard REST calls. There is no need for client-to-server streaming. SSE handles this pattern with a simpler protocol than WebSockets — no upgrade handshake, no framing protocol, no heartbeat management. Spring's `SseEmitter` requires approximately 20 lines of code to implement. WebSocket would require a STOMP broker configuration, a message destination mapping, and client-side subscription management — significantly more complexity for no benefit.

The only scenario where WebSocket would be preferable is if the client needed to send high-frequency streaming data to the server (for example, real-time position updates). This project has no such requirement.

Communication Flow

Operator clicks "Step"

- Frontend: POST /api/simulation/step
- Backend: `SimulationController.step()`
- `SimulationEngine.step()`
- `EventDispatcher` processes next event
- `MutationApplier` updates state
- `SnapshotFactory.create()` → `StateSnapshotDto`
- `SseController.emit(snapshot)` → SSE push
- Frontend: `EventSource` listener fires
- `React setState(newSnapshot)`
- All subscribed components re-render

Operator clicks "Run"

- Frontend: `POST /api/simulation/run { ticks: 100 }`
- Backend: runs `step()` in a loop
- After each step: SSE push (same as above)
- Frontend: UI updates at SSE arrival rate
- At tick 100: SSE push with `simulationComplete` event
- Frontend: `ReportModal` renders

Reconnection Handling

If the SSE connection drops (network interruption, browser tab hidden), the frontend automatically reconnects using the browser's built-in `EventSource` reconnection. On reconnection, the frontend calls `GET /api/simulation/state` to retrieve the current state snapshot and re-renders from that point. No state is lost — the simulation state lives on the server.

13. Technology Recommendations and Justifications

Java 21 + Spring Boot 3

Java 21 provides record classes (used for value objects and DTOs), sealed classes (used for the `Mutation` type hierarchy), and pattern matching (used in the event dispatcher's type-dispatch logic). These language features make the implementation cleaner and more type-safe than older Java versions.

Spring Boot 3 provides auto-configuration, dependency injection for algorithm module registration, `SseEmitter` for SSE streaming, and a mature testing ecosystem (`@SpringBootTest`, `MockMvc`). Spring Web's REST controller model is appropriate for the API surface. Spring's `@Configuration` classes enable clean algorithm module registration without reflection.

Alternative: Python + FastAPI. Valid if Java skills are weak. FastAPI's async SSE support is clean. Algorithm implementations in Python are faster to write but 5–10x slower to execute at equivalent input sizes. At the committed scale, this is acceptable.

React 18 + Vite + Tailwind CSS

React 18's automatic batching and concurrent features improve rendering performance for rapid state updates from SSE. Vite's development server starts in under 1 second and supports HMR, making frontend iteration fast. Tailwind's utility classes eliminate the need to write custom CSS for layout, spacing, and color, saving significant development time.

Alternative: Vue 3 + Vite. Technically equivalent. Choose React because it aligns with existing team experience.

Chart.js 4

Chart.js produces production-quality time-series line charts and bar charts with minimal configuration. It integrates cleanly with React via canvas refs. Its `update('none')` mode enables real-time streaming data updates without performance-degrading animations. The Chart.js bundle size (200KB) is acceptable for a project of this scope.

Alternative: Recharts (React-native chart library). Easier to integrate but has less fine-grained control over update behavior and canvas rendering.

HTML Canvas for Map

Canvas imperative drawing is faster than SVG DOM manipulation for a graph that updates every simulation tick. SVG would create a DOM node per graph element (30 nodes + 70 edges + 6 courier markers = 106+ DOM nodes, all updating each tick). Canvas redraws the entire map in one paint operation. At the map's complexity level, Canvas completes a full redraw in under 5ms.

Alternative: Leaflet.js (with real map tiles). Adds geographic authenticity but requires an internet connection, adds a large dependency, and the tile-loading complexity adds nothing to the algorithmic demonstration.

JUnit 5 + AssertJ

JUnit 5's parameterized tests are used for algorithm correctness tests — the same test runs with multiple known input/output pairs. AssertJ's fluent assertion API produces readable test failure messages that identify exactly what was wrong without deciphering generic assertion error messages.

Docker Compose

Two containers: Spring Boot backend and Nginx serving the React static build. The compose file mounts the `reports/` directory as a volume so generated report files survive container restarts. A `Makefile` with three targets (build, up, down) provides a one-command setup.

Deployment

Vercel for the React frontend: Vercel's CI/CD automatically builds and deploys on every push to the main branch. No configuration required beyond adding a `vercel.json` with the build command and output directory. The deployed frontend must have the backend URL configured via environment variable.

Railway for the Spring Boot backend: Railway supports Java Spring Boot applications natively, detects the project type automatically, and provides a PostgreSQL addon (not needed here, but demonstrates the platform's generality). Railway's free tier is sufficient for demonstration and evaluation.

For local-only demonstration (recommended for demo day): Docker Compose on a dedicated laptop guarantees no network dependency and no cold-start latency.

14. Final Recommended Tech Stack

FRONTEND

Framework: React 18
Build tool: Vite 5
Styling: Tailwind CSS 3
Charts: Chart.js 4
Map rendering: HTML Canvas (native browser API)
State management: React Context + useState (no external library)
Communication: EventSource (native SSE), fetch (REST)
Language: TypeScript (for type safety on DTOs)

BACKEND

Language: Java 21
Framework: Spring Boot 3.3
Web layer: Spring Web MVC
SSE: Spring SseEmitter
Serialization: Jackson (Spring Boot default)
Configuration: Spring @Configuration + @Value
Build tool: Maven 3

COMMUNICATION

Client → Server: REST (HTTP/1.1)
Server → Client: Server-Sent Events (SSE)
Format: JSON

STORAGE

Runtime state: Java heap (in-memory)
Report persistence: JSON files (local filesystem)
Database: None

TESTING

Backend unit: JUnit 5 + AssertJ
Backend integration: Spring Boot Test + MockMvc
Frontend: Vitest + React Testing Library (optional)

DEPLOYMENT

Local: Docker Compose (2 containers)
Frontend (cloud): Vercel
Backend (cloud): Railway
Container base: Eclipse Temurin 21 (JRE slim) + Nginx Alpine

DEVELOPER TOOLING

Version control: Git + GitHub
API testing: HTTPie or curl (no Postman required)
Code formatting: Prettier (frontend), Google Java Format (backend)
Code analysis: ESLint (frontend), SpotBugs (backend, optional)

15. Deployment and Infrastructure

Docker Compose Structure

```
yaml
version: '3.9'

services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    volumes:
      - ./reports:/app/reports
    environment:
      - SIMULATION_MAX_TICKS=100
      - SIMULATION_DP_STOP_CAP=10
      - SIMULATION_QUIET_PERIOD_TICKS=5
      - SIMULATION_DP_BUDGET_MS=200

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:80"
    environment:
      - VITE_API_URL=http://localhost:8080
    depends_on:
      - backend
```

Backend Dockerfile

```
dockerfile
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/delivery-optimizer.jar app.jar
RUN mkdir -p /app/reports
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Frontend Dockerfile

```
dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json .
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

CI/CD

For the course deliverable, CI/CD is a `Makefile`:

```
makefile
build:
    docker compose build

up:
    docker compose up -d

down:
    docker compose down

test:
    cd backend && mvn test
    cd frontend && npm test

logs:
    docker compose logs -f backend
```

For cloud deployment, GitHub Actions runs `mvn test` and `npm test` on every push to main. If tests pass, Railway auto-deploys the backend from the GitHub repository and Vercel auto-deploys the frontend.

Environment Management

All configurable parameters are externalized as environment variables with sensible defaults in `application.yml`. No hardcoded values in business logic. The `.env.example` file documents all required variables for local setup.

16. Metrics, Monitoring, and Reporting

Tracked Metrics

Per algorithm call:

algorithm_name: string
triggering_event_type: EventType
input_size: int (stops, orders, or edges depending on algorithm)
execution_time_ms: double
solution_cost: double (route distance, or cargo wasted capacity)
states_evaluated: int (DP only)
improvement_over_baseline: double (DP only: delta vs. shadow route)
approximation_ratio: double (bin packing only)

Per simulation tick:

tick: int
total_fleet_distance: double
average_distance_per_remaining_stop: double (fleet average)
active_couriers: int
orders_delivered: int
orders_pending: int
on_time_count: int
late_count: int
disruption_events_this_tick: int
quiet_period_active: boolean

Per courier (updated each tick):

courier_id: string
current_node: int
remaining_stops: int
active_route_distance: double
shadow_route_distance: double
quality_gap_percent: double
dp_cycles_completed: int
total_dp_distance_saved: double
cargo_utilization_percent: double

End-of-Shift Report Structure

```
ShiftReport {  
  metadata {  
    scenario_name: string  
    run_id: string  
    timestamp: ISO-8601  
    total_ticks: int  
    configuration: SimulationConfig  
  }  
  
  summary {  
    total_orders_delivered: int  
    total_orders: int  
    on_time_delivery_rate: double  
    total_fleet_distance: double  
    total_distance_saved_by_dp: double
```

```

    greedy_only_shadow_distance: double
    dp_improvement_percentage: double
    total_disruption_events: int
    disruption_breakdown: Map<EventType, int>
  }

  algorithm_performance {
    dijkstra: { call_count, avg_time_ms, total_time_ms }
    greedy_insertion: { call_count, avg_time_ms, avg_positions_evaluated,
      avg_insertion_cost }
    dp_reoptimiser: { call_count, avg_time_ms, avg_improvement_km,
      avg_improvement_pct, budget_exceeded_count,
      verification_passed_count }
    bin_packing: {
      first_fit: { avg_wasted_pct, avg_approximation_ratio }
      best_fit: { avg_wasted_pct, avg_approximation_ratio }
      ffd: { avg_wasted_pct, avg_approximation_ratio }
    }
  }

  courier_summaries: List<CourierSummary> {
    courier_id, orders_delivered, total_distance,
    dp_cycles, distance_saved_by_dp, final_cargo_utilization
  }

  dp_verification_log: List<VerificationRecord> {
    courier_id, remaining_stops, dp_distance,
    brute_force_distance, match: boolean, permutations_evaluated
  }
}

```

17. Testing Strategy

Algorithm Unit Tests

Each algorithm module has a dedicated test class. Tests are written before integration (test-driven within the algorithm development phase, not necessarily TDD for the full project).

Dijkstra tests: Test 1: A known 8-node graph with hand-calculated shortest paths. Assert `distMap[source][destination]` matches expected for all source-destination pairs. Test 2: Dijkstra on a disconnected graph — assert unreachable nodes have `distMap` value of `Double.MAX_VALUE`. Test 3: Dijkstra after an edge weight change — assert only the affected paths change, unaffected paths remain identical.

Greedy insertion tests: Test 1: A 3-courier, 3-stop-each configuration with one new order. Hand-calculate the optimal insertion position and assert the module selects it. Test 2: An insertion where all couriers are over capacity — assert the module returns a `NoValidInsertionMutation` rather than throwing. Test 3: Correctness of insertion cost formula — assert `insertion_cost` matches the hand-calculated detour cost for a known input.

DP re-optimizer tests: Test 1: A courier with 4 remaining stops. Run DP and brute-force. Assert identical results. Test 2: A courier with 10 remaining stops. Assert DP completes within 200ms. Test 3: A courier with 11 remaining stops. Assert `DPOutput.isDeferred() == true`. Test 4: Two identical state inputs produce identical output (determinism).

Bin packing tests: Test 1: Known cargo distribution on 4 couriers. Assert FFD produces the expected assignment. Test 2: An over-capacity scenario. Assert no strategy produces an assignment where any courier exceeds capacity. Test 3: All three strategies on the same input — assert FFD produces equal or better approximation ratio than FF.

Integration Tests

Scenario replay tests: Load the two pre-scripted demonstration scenarios and run them to completion. Assert specific final state values: total orders delivered, couriers active at tick 100, final route quality gap percentages. These tests are the regression protection for the event ordering logic.

Event ordering test: Inject a `ROAD_REMOVAL` and a `NEW_ORDER` in the same tick. Assert that Dijkstra runs before greedy insertion (verifiable by asserting that greedy insertion uses the post-closure distance matrix, not the pre-closure one).

Shadow route test: Run a 20-tick simulation. After each DP cycle, assert that the shadow route distance is greater than or equal to the active route distance (the shadow should never improve without DP).

Simulation Determinism Test

Run the same scenario twice. Assert that the complete event log, all metric values, and the final state snapshot are byte-identical. This test validates reproducibility.

18. Risks and Engineering Trade-offs

Technical Risks

Risk: Simulation engine complexity overruns development schedule Probability: High. Impact: Critical. Mitigation: Use a sorted `ArrayList` as the event queue in the first implementation — $O(n \log n)$ insert, correct ordering, simple to implement. Replace with `PriorityQueue` only after the engine is stable. Do not design a perfect event system on day 1.

Risk: DP produces wrong results that pass through undetected Probability: Medium. Impact: High (invalidates the entire comparison report). Mitigation: Brute-force verifier is mandatory, not optional. Write and run it before integrating DP into the simulation engine. Do not proceed to integration until brute-force verification passes on 5 distinct test cases.

Risk: Route quality chart is flat — greedy and DP show no difference Probability: Medium (with random graph and orders). Low (with adversarial design). Mitigation: Use the designed adversarial graph with two geographic clusters. The quality gap is structural and guaranteed with this graph topology regardless of other inputs.

Risk: Bin packing strategies produce identical results Probability: High (with uniform cargo sizes). Low (with bimodal distribution). Mitigation: Fix cargo size distribution at project start: 30% large (0.6–0.7 slot), 70% small (0.15–0.2 slot). Document the distribution choice and its rationale.

Risk: SSE connection drops during demo presentation Probability: Low. Impact: High (disrupts live demonstration). Mitigation: Run the demonstration using the Docker Compose local deployment, not the cloud-deployed version. The local version has no network dependency and no cold-start latency.

What Should NOT Be Over-Engineered

The event queue implementation: a sorted `ArrayList` is sufficient. Implementing a Fibonacci heap or a custom priority queue is a premature optimization that costs days of work with no observable benefit at 200-event queue depth.

The distance matrix data structure: a 2D double array is correct. A sparse matrix, a graph database, or a custom hash structure adds complexity with no performance benefit at 30×30 scale.

The map rendering: static node positions, straight polylines, no animation. Adding smooth interpolation, curve fitting, or geographic projection is a week of frontend work with zero algorithmic value.

The state snapshot serialization: Jackson's default JSON serialization is sufficient. A custom serialization format, binary encoding, or compression adds complexity with no benefit for a 30KB JSON payload.

Where Simplicity is Preferable

Algorithm module registration: use Spring's `@Component` and `@Qualifier` annotations. Do not build a plugin registry or a reflection-based module loader.

Scenario file format: plain JSON. Do not invent a domain-specific language for scenario definition.

Report generation: use Jackson to serialize the `ShiftReport` object directly. Do not build a PDF generator or a template engine.

19. Optional Features and Future Extensions

Realistic Optional Features (Add Only If Core Is Complete by Day 8)

Algorithm strategy toggle panel A UI toggle that switches which algorithm handles new order events: Greedy Only, DP Only, or Adaptive. Runs the same scripted scenario under all three modes and compares end-of-shift metrics. This is the highest-value optional feature because it directly demonstrates the course's core learning objective — comparing exact vs. approximate algorithms — with a one-click UI. Implementation cost: approximately 4 hours (one mode flag in the dispatcher, one comparison panel in the UI).

Cargo manifest panel Per-courier breakdown showing loaded orders, weights, and slot assignments with a visual slot-fill diagram. Updates after every bin packing event. Implementation cost: approximately 3 hours (one new UI panel, no backend changes).

Configurable parameter panel UI sliders for: courier count, DP stop cap, quiet-period threshold, simulation speed, cargo size distribution parameters. Changes take effect on next simulation reset. This enables live experimentation during demonstrations. Implementation cost: approximately 4 hours (all parameters are already externalized as config values; the UI panel reads and writes them via a REST endpoint).

Advanced Research Extensions (Deferred — Not for This Project)

Real geographic data via OpenStreetMap. Would require geographic coordinate transformation, tile server integration, and routing algorithm adaptation for real road networks. This is a month of work beyond the current scope.

Machine learning demand prediction. Would predict order arrival times to pre-position couriers. Requires training data, a model serving infrastructure, and changes to the simulation's order generation model. Out of scope.

Multi-depot routing. Would require changes to the graph model, the DP state definition, and the courier initialization logic. Architecturally feasible but doubles the implementation complexity.

Features Intentionally Deferred

Vehicle speed variation by road type, turn-by-turn navigation instructions, real-time GPS position tracking, customer-facing delivery tracking UI, and order splitting across multiple couriers are all intentionally deferred. Each adds implementation complexity without adding algorithmic depth — they make the system more realistic without making the algorithm comparison more meaningful.

20. Final Recommended Development Roadmap

Phase 1 — Foundation (Days 1–3)

Day 1: Domain model + graph + Dijkstra Define all value objects (`NodeId`, `CourierId`, `Distance`, `SimulatedTick`). Define all domain classes (`Node`, `Edge`, `Order`, `Courier`, `Route`, `Stop`, `CargoItem`). Build `GraphStore` with an adjacency list structure. Implement `DijkstraModule` as a pure function. Write 3 Dijkstra unit tests with hand-calculated expected values. Pre-compute the 30×30 distance matrix. All tests pass before proceeding to day 2.

Day 2: Simulation engine skeleton + event queue Define `Event`, `EventType`, `EventPriority`. Implement the event queue as a sorted `ArrayList` with comparator. Implement `SimClock`. Implement `SimulationEngine` with `step()` method that dequeues one event and logs it (no algorithm dispatch yet). Implement `MutationApplier` with no-op mutation handling. Verify tick-by-tick progression produces the correct event ordering on a 10-event test queue.

Day 3: Disruption events (road changes) + state layer Implement `GraphStore` edge mutation operations. Implement `ROAD_WEIGHT_CHANGE` and `ROAD_REMOVAL` event handlers in `EventDispatcher`. Wire Dijkstra re-computation into these handlers. Implement `StateEmitter` producing a basic state snapshot. Build the Spring Boot application with one endpoint (`POST /api/simulation/step`). Verify via `curl` that stepping through road events updates the distance matrix correctly.

Phase 2 — Core Algorithms (Days 4–6)

Day 4: Greedy insertion + order registry + shadow routes Implement `GreedyInsertionModule` with correct $O(k \times m)$ complexity. Implement `OrderRegistry` with pending/assigned/delivered states. Implement `ShadowRouteRegistry` — initialized alongside route registry, receives all insertions. Connect greedy insertion to `NEW_ORDER` event. Write 3 greedy insertion unit tests. Verify on a 5-order, 3-courier manual test case.

Day 5: Bin packing — all three strategies Implement `FirstFitStrategy`, `BestFitStrategy`, `FirstFitDecreasingStrategy`, each implementing a common `PackingStrategy` interface. Implement `BinPackingModule` that executes all three strategies on each redistribution event and returns a `BinPackingOutput` containing all three results. Connect to `NEW_ORDER` capacity validation and `VEHICLE_BREAKDOWN` redistribution. Write bin packing unit tests with bimodal cargo distribution.

Day 6: DP re-optimizer + brute-force verifier Implement `DPreoptimiserModule` using the held-karp formulation. Implement `BruteForceVerifier` for $n \leq 5$. Run verifier against DP on 5 distinct test cases — all must agree. Connect DP to `QUIET_PERIOD` event via `QuietPeriodMonitor`. Implement `MetricsStore` with per-algorithm metrics logging. All algorithm unit tests passing before day 7.

Phase 3 — Integration + Metrics (Days 7–8)

Day 7: Full simulation integration + metrics layer Connect all event handlers in `EventDispatcher`. Run a full 100-tick scripted scenario end-to-end. Verify all state transitions produce correct output. Implement `SnapshotFactory` producing complete `SystemStateSnapshot`. Implement SSE endpoint. Verify SSE push delivers state snapshots correctly.

Day 8: Metrics collection + report generation Implement `MetricsStore` tick-by-tick snapshot accumulation. Implement `ReportGenerator` producing complete `ShiftReport`. Implement `GET /api/reports/shift` endpoint. Run a full scenario and verify the report contains all required fields with sensible values. Fix any data flow issues found.

Phase 4 — Frontend (Days 7–9, parallel with backend)

Day 7 (frontend): React project setup + SSE connection Initialize Vite + React + Tailwind. Implement `SimulationProvider` with SSE connection. Implement `ControlPanel` with Step and Reset buttons connected to REST endpoints. Verify that clicking Step updates React state from SSE response.

Day 8 (frontend): Map canvas + event log Implement `DeliveryMapCanvas` with full graph rendering. Verify map redraws correctly after each tick. Implement `EventLogPanel` with virtualized scrolling.

Day 9 (frontend): Charts + metrics panels + report modal Implement `RouteQualityChart` with DP-improved and shadow time-series. Implement `GreedyVsDPPanel`. Implement `BinPackingPanel` with slot-fill bars. Implement `ReportModal` triggered by `simulationComplete` SSE event.

Phase 5 — Scenarios + Polish + Delivery (Day 10)

Day 10: Scripted scenarios + README + buffer Implement the two pre-scripted demonstration scenarios as `ScenarioDefinition` JSON files. Verify both run end-to-end and produce visually distinct algorithm behaviors. Write README covering setup, usage, algorithm descriptions, and complexity analysis. Write algorithm comparison report (required course deliverable). Export one full simulation run's report as the project's demonstration artifact. This day is also the buffer for any slippage from days 6 or 8. Do not add new features on day 10.

End of Project Blueprint